Due Date:5/6/2026 **March 9, 2025**

EE 433Real-time Applications of Digital Signal Processing

Active Noise Cancellation

In this project, you will apply active noise technology for noise and interference cancellation in real-time. You will use microphone/microphones to receive noise in close proximity and suppress it in the headphone. Active Noise Cancellation (ANC) is the process of suppressing unwanted noise and interference by adding a 180 degrees out of phase version at the cancellation point. There are two cases for the project which require single and two microphone use.

Case 1: External microphone (mic) picks up the noise signal away from the headphone. The separation between mic and the headphone determines the processing time allowed before the wave reaches to ear. Processing estimates the inverse waveform and it is played with the desired signal so that only desired signal (such as music, speech) is heard in the headphone. This is a feedforward structure.

Case 2: The function of the first mic is to capture the noise before the wave reaches to the headphone as in Case 1 and predict the inverse waveform for cancellation. The second mic is placed inside one of the headphone to measure the waveform heard by the ear to generate a feedback channel. It is used to compute an error signal which is the residual noise. This error is minimized to improve the quality.

Figure 1 shows the components that will be used in this project.

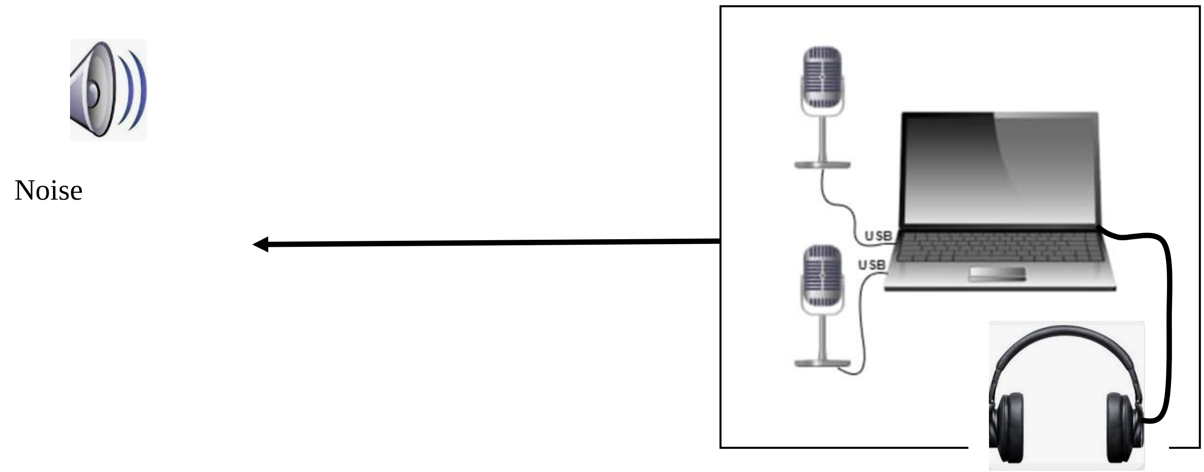

Figure 1. Two microphones connected to a computer together with a headphone

In order to receive two microphone signals simultaneously, you can use the following Y cable or module to convert two mono microphone signal to a stereo signal and sample the stereo signal simultaneously. Examples of such items are given in Figure 2.

Figure 2. Stereo breakout cable and module.

Your program should be able to plot both time and frequency plots of the input signals in real-time. User interface and plots should be implemented in LabVIEW. All the processing will use C/C++. Hence your C++ code will be called from LabVIEW as a share library object. User interface should have all the relevant parameters to have full control of your task (one of which is sampling frequency). Note that your signal processing should be very efficient and fast in order to get reliable results in real-time. You are free to use any Signal Processing technique you learn in this course and beyond. Some examples of such methods are Adaptive Filtering (LMS, NLMS, FxLMS), Wiener Filtering, FIR, IIR filter implementations, Spectral Analysis (time-frequency representation), Noise prediction, etc.

1st Part (Due: April 26, 2026): (%40 of the total grade)

In this part, you will implement Case 1 and Case 2 offline. In other words, you can record and process later. In this case, you should be able to show that you can get noise suppression better than 10dB. This should be shown by plotting the received and cleaned signals, least-squares error (LSE) at the output and output SNR, etc. so that there are objective reasons to believe that the task is accomplished. Note that the suppression may depend on the noise characteristics (such as narrowband, wideband, coherent, etc.). You can show your performance for different types by using natural and simulated noise sources (like engine noise (natural, wideband), single tone (sine), etc.). The richness and the number of noise types improve your grade in this section. The programming type can be chosen as you wish namely you can program everything in MATLAB for this part only. You need to upload your report and codes to ODTUClass for due date.

2nd Part (Due: June 5, 2026): (%60 of the total grade)

In the second part of the project, you will implement the Case 1 and 2 in real-time in C/C++ and LabVIEW. User interface should allow any parameter choice and its change in real-time. All the relevant plots should be plotted in real-time to evaluate the quality of the processing and output response. The complete project (Task1 and Task2) will be evaluated on final due date in EE433 Laboratory. The report and codes should be uploaded to ODTUClass before demo session.

A shared library file “my.dll” can be generated by compiling your “my.c” file in Windows. LabVIEW can be used for accessing the microphones and feeding the signal samples to the C/C++ program and for generating the user interface.

The main function of the LabVIEW program in this project is to call a shared library which is coded in C/C++ and generate the user interface. Since the program is to be run on your computer (assuming Windows operating system), it should be compiled by LabWindows CVI. Otherwise there may be incompatibility between LabVIEW and the “*.dll” shared library. Once you have obtained the .dll file in your computer, you can call it in LabVIEW by appropriately defining the parameters, inputs, etc. The details of running a C/C++ code under LabVIEW is given in the following references. You will submit your written project report together with your C/C++ code and LabVIEW codes.

1) Integrating C Code with LabVIEW on NI Linux Real-Time Targets

http://www.ni.com/tutorial/14690/en/

2) Getting Started with C/C++ Development Tools for NI Linux Real-Time, Eclipse Edition.

http://www.ni.com/tutorial/14625/en/

3)https://www.youtube.com/watch?v=CdG--UMpNhg Video.

4) http://www.ni.com/download/labview-myrio-toolkit-2018/7583/en/
Compiler settings

4) An Overview of Accessing DLLs or Shared Libraries from LabVIEW-DevZone Article

5) Example 1: Call a Shared Library That You Built

http://zone.ni.com/reference/en-XX/help/371361L-01/lvexcodeconcepts/ex_1_call_a_shared_library/

6) LabWindow/CVI Programmer Reference Manual

7) Using LabWindows/CVI DLLs in LabVIEW Real-Time ... - LabVIEW360

Note: The project demo will be scheduled for each group. You will bring your printed project report at the project demo session and submit your code and report to ODTUClass before the deadline.